# LLM from Scratch — Session 03

**날짜:** 2026년 9월 5일

**주제:** 손상 코퍼스 교체 · Raschka 2장 재독과 재타이핑 (2.1 ~ 2.4)

**직전 세션:** LLM_project_session_02.md (2026.9.3)

**커밋:** `e133cff`

---

## 0. 이 세션의 성격

2장을 **두 번째로** 읽으면서 코드를 처음부터 다시 타이핑했다. 세션 02에서 만든 `src/tokenizer.py`는 그대로 두고, 빈 파일에서 다시 쳤다.

두 번째 읽기가 첫 번째보다 이해가 잘 된다는 것은 사실이지만, 오늘 나온 버그 다섯 개는 **읽어서 아는 것과 쳐서 되는 것이 다르다**는 것을 그대로 보여준다. 이해도가 올라간 것은 맞고, 그것이 오타를 막아주지는 않는다. 두 진술은 모순되지 않는다.

---

## 1. 세션 02 미완료 항목 처리

### `the-verdict.txt` 재취득 — 완료

세션 02의 "다음 세션 첫 작업"이었다. 재다운로드가 필요 없었다. 이미 받아둔 Raschka 공식 저장소(`docs/LLMs-from-scratch-main/`)에 정본이 들어 있었다.

| 파일 | 문자 | 토큰 | 어휘 | 백슬래시 |
|---|---|---|---|---|
| 교체 전 | 20,479 | 4,772 | 1,131 | 82 |
| **교체 후** | **20,479** | **4,690** | **1,130** | **0** |
| 책 기준값 | 20,479 | 4,690 | 1,130 | 0 |

**손상본을 지우지 않고 `data/the-verdict.corrupted.txt`로 남겼다.** 이유:

> 길이가 정확히 같은데(20,479자) 어휘 항목이 정확히 하나 더 많은 코퍼스 쌍이 손에 있다. 12월 데이터 포이즈닝 실험에서 "오염 전 / 오염 후"의 대조군으로 이보다 깨끗한 실물 표본을 일부러 만들기 어렵다. 오염이 0.0004%(20,479자 중 82자)이고, 그 결과가 어휘 1개 증가라는 것까지 이미 측정되어 있다.

### `Untitled.ipynb` — 미처리

`a1fc822`에 커밋되어 있어 `git rm --cached`가 필요하다. 여전히 판단 대기.

---

## 2. 오늘 만난 버그 — 두 가지 유형뿐이다

다섯 개가 나왔지만 실제로는 **두 종류**다. 이 구분이 오늘의 핵심이다.

### 유형 A — 에러를 내지 않고 조용히 결과를 바꾸는 것

셋 다 눈에 보이지 않는 문자 하나가 원인이고, 셋 다 파이썬이 아무 불평도 하지 않는다.

#### A-1. `''` 를 `"` 로 착각

```
re.split(r'([,.:;?_!''()\']|--|\s)', raw_text)   # 홑따옴표 두 개
re.split(r'([,.:;?_!"()\']|--|\s)', raw_text)    # 책 — 쌍따옴표 한 개
```

파이썬은 `r'([,.:;?_!'` 와 `'()\']|--|\s)'` 를 **붙어 있는 문자열 두 개**로 읽고 이어붙인다. 결과는 `([,.:;?_!()']|--|\s)` — 문자 클래스에서 `"` 만 사라진다.

문법 오류가 아니다. 정규식으로도 유효하다. 다만 **다른 정규식**이다.

| | 토큰 수 |
|---|---|
| 사용자 패턴 | 4,606 |
| 책 패턴 | 4,690 |

#### A-2. `decode` 정규식의 뒤쪽 `\s`

```
re.sub(r'\s([,.:;?_"!()\']|--)\s', r'\1', text)   # 뒤에도 \s
re.sub(r'\s+([,.:;?_!"()\'])', r'\1', text)       # 책 — 앞 공백만
```

뒤의 `\s` 까지 매칭해서 소비하므로 구두점 **뒤** 공백이 사라진다.

```
join      : '" It \' s the last he painted , you know , " Mrs . Gisburn ... pride .'
사용자     : '" It\'s the last he painted,you know," Mrs.Gisburn ... pride .'
책        : '" It\' s the last he painted, you know," Mrs. Gisburn ... pride.'
```

`painted,you` , `Mrs.Gisburn` 처럼 붙어버리고, 맨 끝 `pride .` 은 마침표 뒤에 공백이 없어 매칭 자체가 안 된다.

#### A-3. `join` 의 공백

```
"<|endoftext|>".join((text1, text2))     # 공백 없음
" <|endoftext|> ".join((text1, text2))   # 책
```

| | ID 열 |
|---|---|
| 공백 없음 | `[..., 975, 10, 1131, 988, ...]` ← 1131 = `<\|unk\|>` |
| 공백 있음 | `[..., 975, 10, 1130, 55, 988, ...]` ← 1130 = `<\|endoftext\|>` |

공백이 없으면 정규식이 `?` 에서 자른 뒤 `<|endoftext|>In` 이 한 덩어리로 남는다. 어휘에 없으므로 `<|unk|>`가 된다. **문서 경계를 표시하려고 넣은 특수 토큰이 "모르는 단어"로 처리되어 사라진다.** `In`도 함께 삼켜진다.

> **유형 A의 공통점:** 셋 다 실행은 정상이고, 출력도 그럴듯하고, 에러도 없다. **책이 적어둔 숫자와 대조하는 것 말고는 발견할 방법이 없다.** 세션 02에서 손상 코퍼스를 두고 적은 "출력이 그럴듯하다는 것은 맞다는 뜻이 아니다"가 하루 만에 세 번 반복되었다.

### 유형 B — 이 이름이 어느 파일에 사는가

이것도 셋인데 **전부 같은 하나**다.

```
NameError: name 'vocab' is not defined
NameError: name 'preprocessed' is not defined
__init__(self, text) 인데 몸통에서 바깥의 vocab 을 사용
```

세 번째가 가장 중요하다.

```
class SimpleTokenizerV1:
    def __init__(self, text):       # 받아놓고 안 쓴다
        self.str_to_int = vocab     # 대신 바깥 전역변수를 훔쳐본다
```

**같은 파일에 있는 동안은 우연히 돌아간다.** 클래스를 `SimpleTokenizerV1.py`로 분리하는 순간 죽는다. 그 파일에는 `vocab`이 없기 때문이다.

책이 `def __init__(self, vocab)`으로 쓴 이유가 이것이다. **받아야 옮길 수 있다.** 클래스가 바깥을 훔쳐보면 그 클래스는 태어난 파일 밖으로 나갈 수 없다.

### 기타

- `encode`에 `return ids` 누락 — 파이썬은 `None`을 반환하고 에러를 내지 않는다
- `tokenizer` / `tokennizer`, `preprocessed` / `prepprocessed` 오타
- **버그 아님:** `[,.:;?_"!()\']` 와 `[,.:;?_!"()\']` — 문자 클래스 안에서 순서는 의미가 없다

---

## 3. 아직 붙지 않은 것 — import 과 파일 간의 관계

본인 표현: *"파일관리, 디렉토리 관리, 파일들 간의 관계 등이 어렵네요."*

이것은 **미해결(버그)이 아니라 미숙지**다. 구분해서 기록해 둔다.

### 붙잡고 갈 문장

> **`import`는 "그 파일을 처음부터 끝까지 한 번 실행하고, 거기 생긴 이름을 쓰겠다"는 뜻이다.**

`from raw_text import vocab` 은 `vocab` 하나만 데려오는 것이 아니다. `raw_text.py` 전체가 실행된다. `print` 도 `urlretrieve` 도 전부 돈다. 그래서 다른 파일을 실행했는데 `raw_text.py` 의 출력이 딸려 나오고 코퍼스를 다시 받는다.

막는 장치가 `if __name__ == "__main__":` 이다.

| 실행 방식 | `__name__` 값 | `if` 블록 |
|---|---|---|
| `python raw_text.py` | `"__main__"` | 실행됨 |
| 다른 파일이 `import raw_text` | `"raw_text"` | 실행 안 됨 |

**"보여주는 코드"와 "제공하는 코드"의 분리.** `src/` 에 들어갈 모든 파일이 이 형태가 된다.

### 폴더의 역할

```
data/       재료.     코드 아님. 읽기만. 커밋 안 됨(.gitignore)
notebooks/  실험실.   지저분해도 됨. 시행착오, 출력 확인, 버릴 코드
src/        도구상자. 실험실에서 살아남은 것만. 깨끗한 함수와 클래스
tests/      검사기.   src/의 도구가 여전히 작동하는지 자동 확인
```

흐름은 **노트북 → src → tests 한 방향**이다. 역방향은 없다.

### 경로

```
file_path = "the-verdict.txt"
```

이것은 "이 파일이 어디 있는지"가 아니라 **"내가 지금 서 있는 폴더 기준"** 이라는 뜻이다. 실행 위치가 바뀌면 가리키는 곳이 바뀐다. `data/` 에 이미 있는데도 루트에 또 받아진 이유가 이것이다.

**규칙: 항상 저장소 루트에서 실행하고, 경로는 루트 기준으로 쓴다.**

### 판단

2.4~2.6절에서 같은 상황을 서너 번 더 만나야 붙을 종류다. **지금 붙들고 있는 것보다 진도를 나가는 것이 맞다고 판단했다.** 이 판단 자체는 옳다. 다만 3장에 들어가기 전에는 한 번 정리하고 가야 한다 — 그때는 파일이 다섯 개가 아니라 열 개가 되어 있다.

---

## 4. 확인된 사실 — 토큰화는 정보를 잃는다

오늘 이것을 두 번, 서로 다른 크기로 목격했다.

### 작은 손실 — 공백 하나

```
원문   : "It's the last he painted, you know," ...
복원   : " It' s the last he painted, you know," ...
```

`It's`가 `It` / `'` / `s` 세 조각으로 쪼개지고, `decode`는 구두점 **앞** 공백만 지우므로 `'` 뒤 공백이 남는다. **책의 코드도 완벽한 왕복을 하지 못한다.** 버그가 아니라 이 설계의 한계다.

### 큰 손실 — 단어 통째로

```
원문   : Hello, do you like tea? <|endoftext|> In the sunlit terraces of the palace.
복원   : <|unk|>, do you like tea? <|endoftext|> In the sunlit terraces of the <|unk|>.
```

`Hello`와 `palace`가 **둘 다 ID 1131**이다. 서로 다른 두 단어가 같은 숫자가 되었고, 되돌릴 방법이 없다. 어휘 밖은 전부 구별 불가능해진다.

> **이것이 2.5절 BPE의 동기다.** 모르는 단어를 `<|unk|>`로 버리는 대신 **아는 조각으로 쪼갠다.** `palace`를 통째로 잃는 대신 조각으로 표현하면 손실이 크게 줄어든다.

### 어휘 ID에는 의미가 없다

```
'!' → 0     'A' → 11    'I' → 53    'a' → 115    'the' → 988
```

`sorted()`는 유니코드 코드포인트 순으로 정렬한다. 구두점(33~63)이 대문자(65~90)보다, 대문자가 소문자(97~122)보다 앞이다. 그래서 `'Zebra'`가 `'apple'`보다 앞에 온다.

**`'a'`가 115번인 데는 아무 의미가 없다.** 114번, 116번과 의미상 아무 관계가 없다. 그런데 이 숫자를 그대로 신경망에 넣는다. 신경망 입장에서 115는 114보다 큰 수다.

**의미 없는 순서에 수치적 크기가 부여되는 문제** — 2.7절 임베딩이 답하는 질문이다. ID는 주소일 뿐이고, 내용물은 학습으로 따로 정한다.

### 특수 토큰의 위치

```
('yourself', 1129)
('<|endoftext|>', 1130)
('<|unk|>', 1131)
```

`sorted()` 다음에 `extend()`를 했으므로 맨 끝에 붙는다. 정렬에 포함시켰다면 `<`의 코드포인트가 60이라 구두점 근처에 끼어들었을 것이다.

순서 자체에 의미는 없지만, **어휘를 나중에 늘려도 기존 ID가 바뀌지 않는다**는 성질은 중요하다.

---

## 5. 오늘 검증된 숫자

다음 세션에서 무언가 어긋나면 이 표부터 대조할 것.

| 항목 | 값 |
| --- | --- |
| 문자 수 | 20,479 |
| 토큰 수 | 4,690 |
| 어휘 (특수토큰 없음) | 1,130 |
| 어휘 (특수토큰 포함) | 1,132 |
| `<\|endoftext\|>` ID | 1,130 |
| `<\|unk\|>` ID | 1,131 |
| V1 테스트 문장 ID 개수 | 21 |
| V2 테스트 ID 열 | `[1131, 5, 355, 1126, 628, 975, 10, 1130, 55, 988, 956, 984, 722, 988, 1131, 7]` |

---

## 6. 저장소 정리 — 이번 세션 처리분

### 커밋 `e133cff`

포함:

```
.gitignore              루트 코퍼스 사본, Raschka 저장소 원본 제외 규칙 추가
.vscode/settings.json   신규
NOTES.md                세션 03 기록
raw_text.py             신규 (루트)
SimpleTokenizerV2.py    신규 (루트)
```

제외:

| 대상 | 처리 | 이유 |
| --- | --- | --- |
| `/the-verdict.txt` (루트) | `.gitignore` | `data/` 에 정본이 있다. `raw_text.py` 가 실행할 때마다 루트에 다시 받는다 |
| `docs/LLMs-from-scratch-main/` | `.gitignore` | 3.9MB. **남의 저장소다.** 내 저장소에 섞이면 "직접 구현했다"는 증거가 흐려진다 |
| `docs/LLMs-from-scratch-main.zip` | `.gitignore` | 2.2MB |
| `token_ID.py` | 스크래치패드로 이동 | `re.split(..., raw_text)` — 모듈 객체를 문자열로 넘기는 중간 산물. 실행되지 않는다 |

### `.vscode/settings.json` — 새로 만든 이유

인터프리터 지정이 VS Code 개인 설정에만 있어서 폴더를 다시 열 때마다 골라야 했다. 이제 자동이다.

```
"python.defaultInterpreterPath": "${workspaceFolder}/.venv/bin/python",
"python.testing.pytestEnabled": true,
"editor.formatOnSave": false
```

**`editor.formatOnSave` 는 일부러 껐다.** 저장할 때마다 편집기가 코드를 정리하면, 손으로 친 것이 손으로 친 그대로 남지 않는다. 세션 01 §6의 첫 번째 습관("오타를 내고 그 오타를 디버깅하면서 배운다")과 정면으로 충돌한다.

---

## 7. 정리안 — 다음 세션에 할 것

**지금 저장소에는 같은 것이 두 벌 있다.** 세션 02의 `src/tokenizer.py`(V1)와 오늘의 `SimpleTokenizerV2.py`(루트, V2)다. 지금은 파일이 적어 문제가 없지만, 3장에서 `attention.py`가 생기면 루트가 뒤섞인다.

### 순서

**1. 실행할 때마다 다운로드하는 것부터 막는다**

`raw_text.py`의 `urlretrieve`가 `if __name__` 밖에 있어 import 할 때마다 실행된다. 12월에 코퍼스를 일부러 오염시켜 실험할 때 무심코 돌리면 **실험 데이터가 덮어써진다.**

```
import os
file_path = "data/the-verdict.txt"
if not os.path.exists(file_path):
    urllib.request.urlretrieve(url, file_path)
```

경로도 `data/`로 바꾼다. 지금은 루트와 `data/`에 같은 파일이 두 벌 있다.

**2. `src/tokenizer.py` 하나로 합친다**

```
src/tokenizer.py
    SPLIT_PATTERN
    preprocess(text)
    build_vocab(text, special_tokens=None)
    SimpleTokenizerV1     # 어휘 밖 → KeyError
    SimpleTokenizerV2     # 어휘 밖 → <|unk|>
```

V1을 지우지 않는다. **둘을 나란히 두는 것 자체가 자료다** — 같은 입력에 하나는 죽고 하나는 살아남는다.

**3. `tests/test_tokenizer.py`를 늘린다**

기존 6개에 V2용을 더한다.

```
def test_v2_maps_unknown_to_unk():
    # V1이 KeyError로 죽던 자리에서 V2는 <|unk|>를 낸다
    ...

def test_endoftext_survives_join_with_spaces():
    # 공백 없이 join 하면 <|endoftext|>가 <|unk|>가 된다 — 오늘 실제로 겪은 버그
    ...
```

두 번째 테스트가 특히 가치 있다. **오늘 실제로 당한 버그를 테스트로 굳히는 것**이고, 이런 테스트는 책에서 베낄 수 없다.

**4. 루트 정리**

`raw_text.py`, `SimpleTokenizerV2.py`를 `notebooks/02_text_data.ipynb`로 흡수하거나 `src/`로 옮긴다. `rm` 대신 `git mv` 또는 `mv`.

5. **`Untitled.ipynb`** — `git rm --cached` 여부 판단.

> **주의:** 1~4는 **2장을 끝낸 뒤에** 한다. 지금 하면 2.5~2.8절 작업이 정리 작업에 끊긴다. 다만 3장에 들어가기 전에는 반드시 한다.

---

## 8. 다음 단계 — 2.5절 BPE

`tiktoken`이 들어온다. **오늘까지와 성격이 다른 절이다.** 직접 구현하지 않고 라이브러리를 쓴다.

### 예고

- `palace`, `Hello` 같은 어휘 밖 단어가 **어떻게 조각나는지** 직접 본다. `<|unk|>`가 사라진다
- `love—`, `wingèd` (세션 02에서 관찰한 em dash 문제)를 BPE로 인코딩해서 조각 수를 센다
- 같은 문장의 한국어판과 영어판 토큰 수를 비교한다. 세션 01 §9의 "2~3배" 주장을 실측으로 검증할 것

### 실행 전 예측 (세션 01 §6-2)

`tiktoken`으로 다음을 인코딩하기 **전에** 조각 수를 적어둘 것.

| 입력 | 예측 | 실측 |
|---|---|---|
| `"palace"` | | |
| `"Hello"` | | |
| `"love—"` | | |
| `"안녕하세요"` | | |

틀리면 거기서 멈추고 이유를 찾는다.

---

## 9. 총평

**잘된 것.**

세션 02가 지목한 "커밋이 없다"가 해결되었다. 오늘 작업은 당일 커밋되었다.

두 번째 읽기를 하면서 **코드를 다시 쳤다.** 첫 번째와 같은 코드를 다시 치는 것은 낭비처럼 보이지만, 오늘 나온 버그 다섯 개가 그 판단이 옳았음을 보여준다. 읽어서 아는 것과 쳐서 되는 것은 다르다. 그리고 이 다섯 개는 다시 치지 않았으면 만나지 않았을 것이고, 만나지 않았으면 배우지도 않았을 것이다.

**모르는 것을 모른다고 말했다.** "아직 정확히 이해하지 못했습니다만 계속 진도를 나가겠습니다"는 정확한 자기진단이고, 동시에 옳은 판단이다. 여기서 붙들고 있었으면 오늘 2.4절까지 못 왔다.

**부족한 것.**

전체 경로를 매번 타이핑하고 있다. `source .venv/bin/activate` 한 번이면 `python 파일명`으로 끝난다. 사소해 보이지만, 매 실행마다 100자를 치는 마찰이 "일단 돌려보자"를 막는다. 세션 01 §6의 두 번째 습관(실행 전 예측)은 **실행이 싸야** 성립한다.

**기록해 둘 것.**

오늘 버그 다섯 개 중 셋이 **눈에 보이지 않는 문자**였다. 따옴표 종류, 공백 하나, 정규식의 `\s` 하나. 파이썬은 셋 다 아무 말 없이 받아들이고 다른 일을 했다.

이것이 이 프로젝트 전체에서 반복될 형태다. **모델은 틀렸다고 말해주지 않는다.** 그럴듯한 출력을 내놓을 뿐이다. 세션 02의 손상 코퍼스, 오늘의 정규식 세 개, 그리고 12월에 할 데이터 포이즈닝 실험이 전부 같은 이야기다 — **검증 기준을 따로 갖고 있지 않으면 틀린 것을 맞은 것으로 안다.**

책이 숫자를 적어둔 이유가 이것이고, 테스트를 쓰는 이유가 이것이다.